

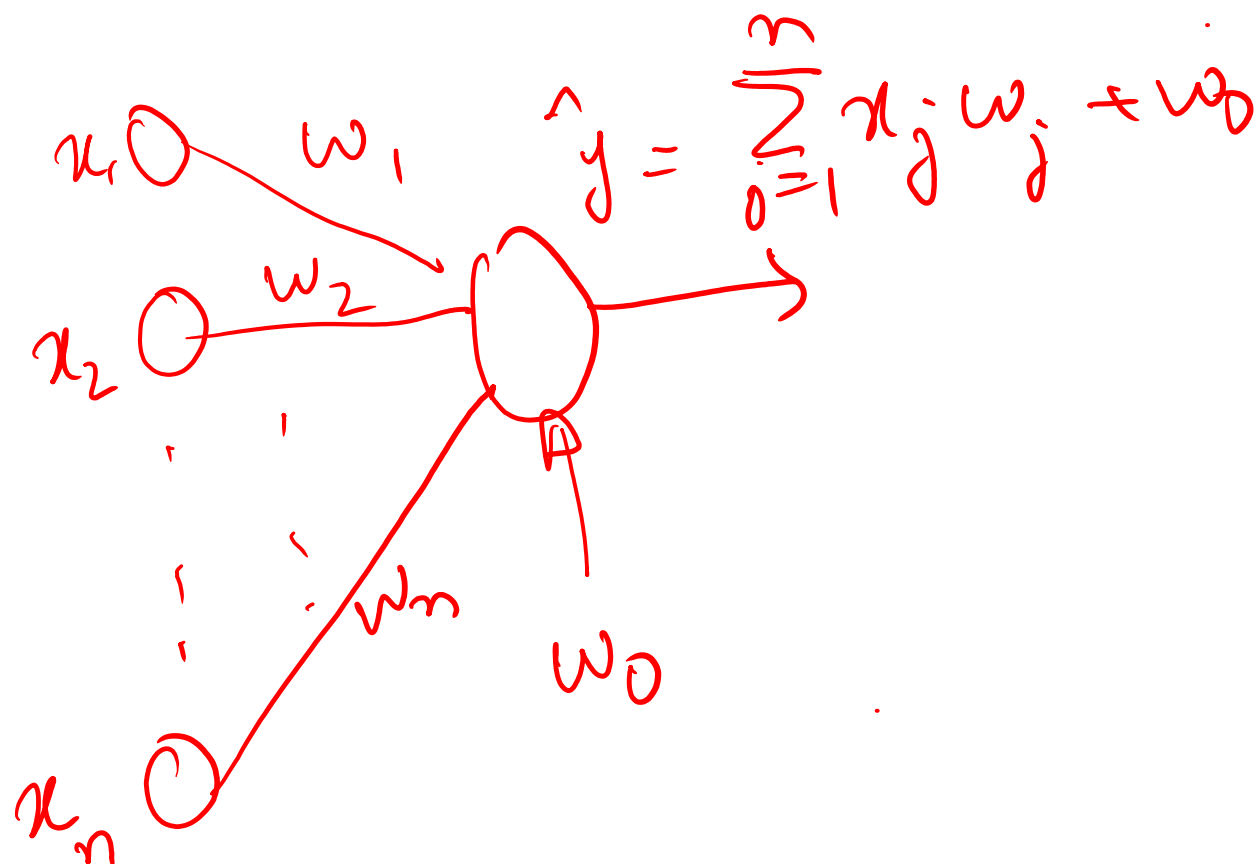
Introduction to Feedforward Networks

Linear models

Regression

$$x \rightarrow y \in \mathbb{R} \quad x \in \mathbb{R}^n$$

$$w \cdot x + w_0 \rightarrow \hat{y}$$

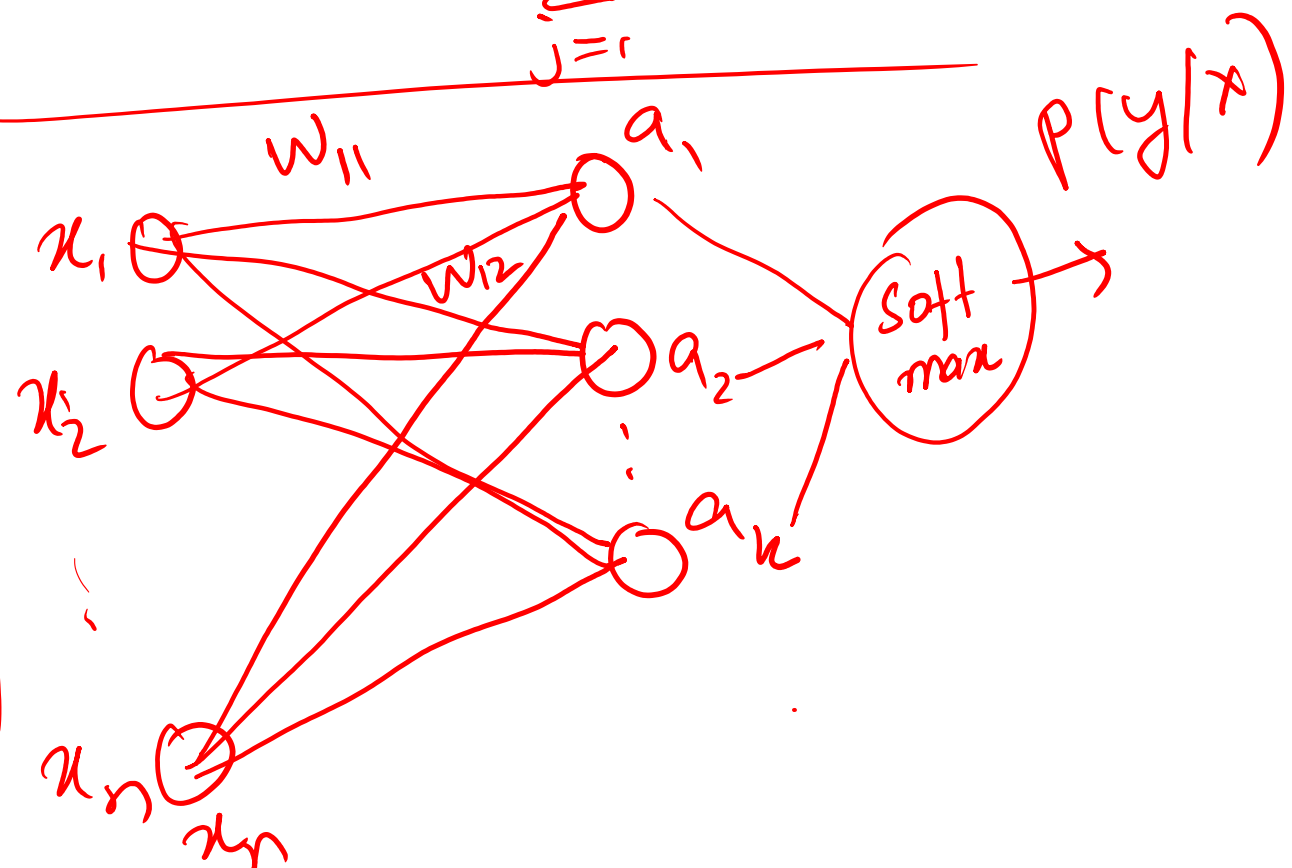


Classification:
 $C \in \{1, \dots, k\}$

$$a_k = w_k \cdot x + w_{k0}$$

$$\hat{y} = \arg \max_k a_k$$

$$P(y|x) = \frac{e^{a_y}}{\sum_{j=1}^k e^{a_j}}$$

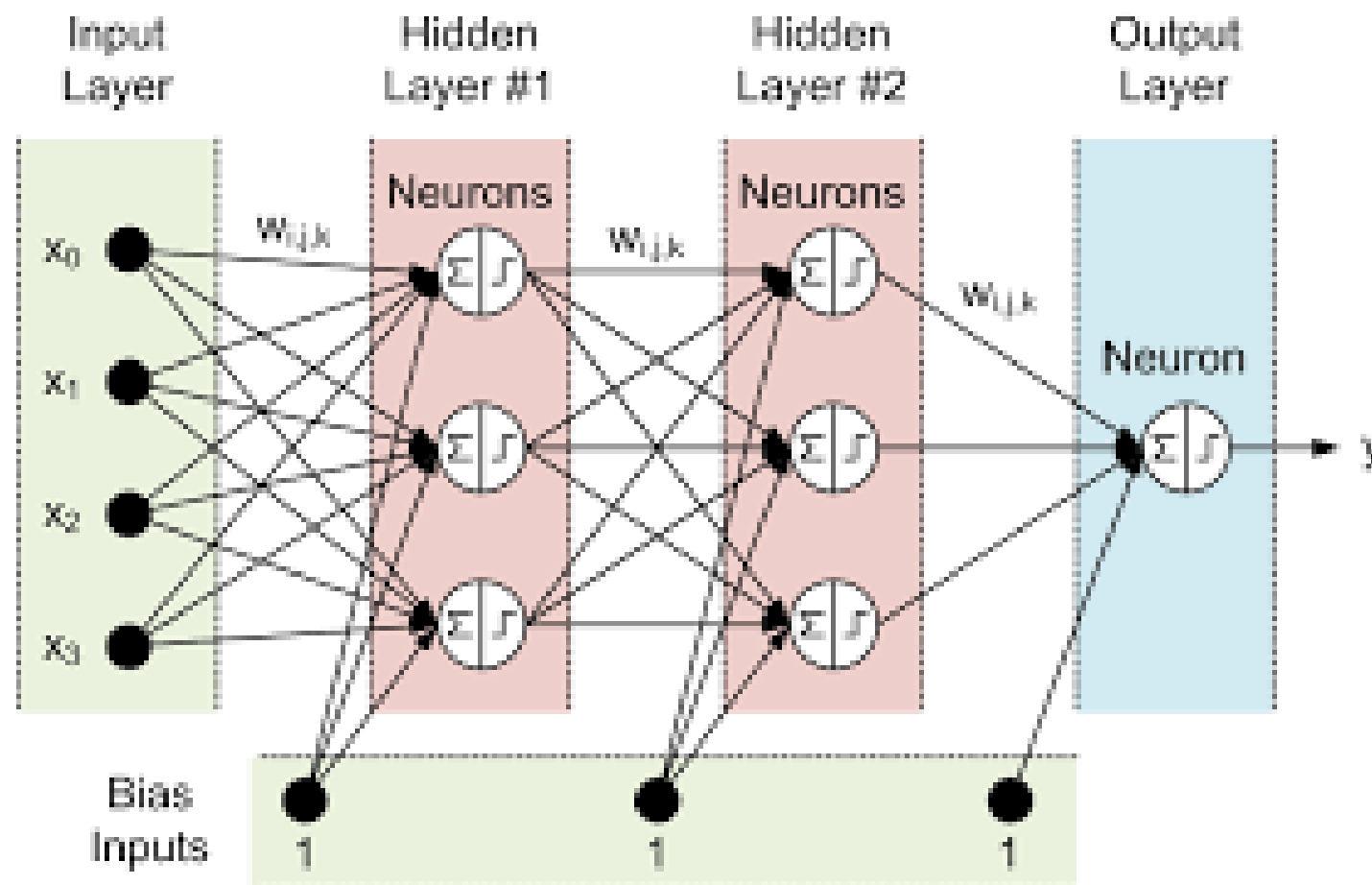


Non-linear functions?

- Embed input into a higher dimensional space
- Example to create a quadratic decision surface design $\phi(x) = [x^2, x, 1]$
$$\hat{a}_j = w_j \phi(x)$$
- Burden on user to design the right embedding.
 - Difficult for complicated data types, e.g. image, speech, time-series, text, etc.
- Use universal kernel, e.g. RBF kernel
 - Expensive to train, at least quadratic in the size of the input.
 - Cannot scale to millions of examples.

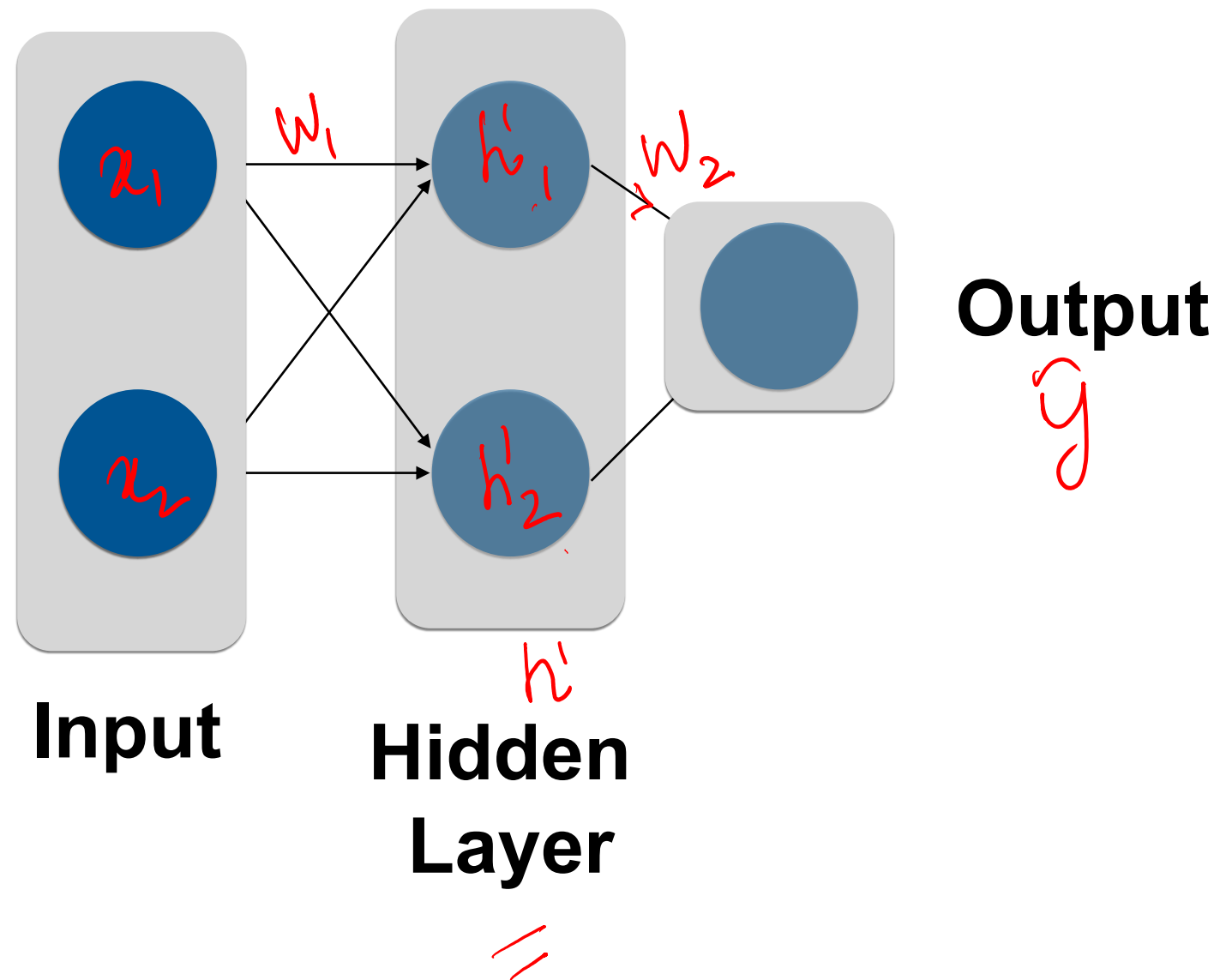
Traditional Neural Networks

Many linear functions connected layer-wise after simple non-linear activation

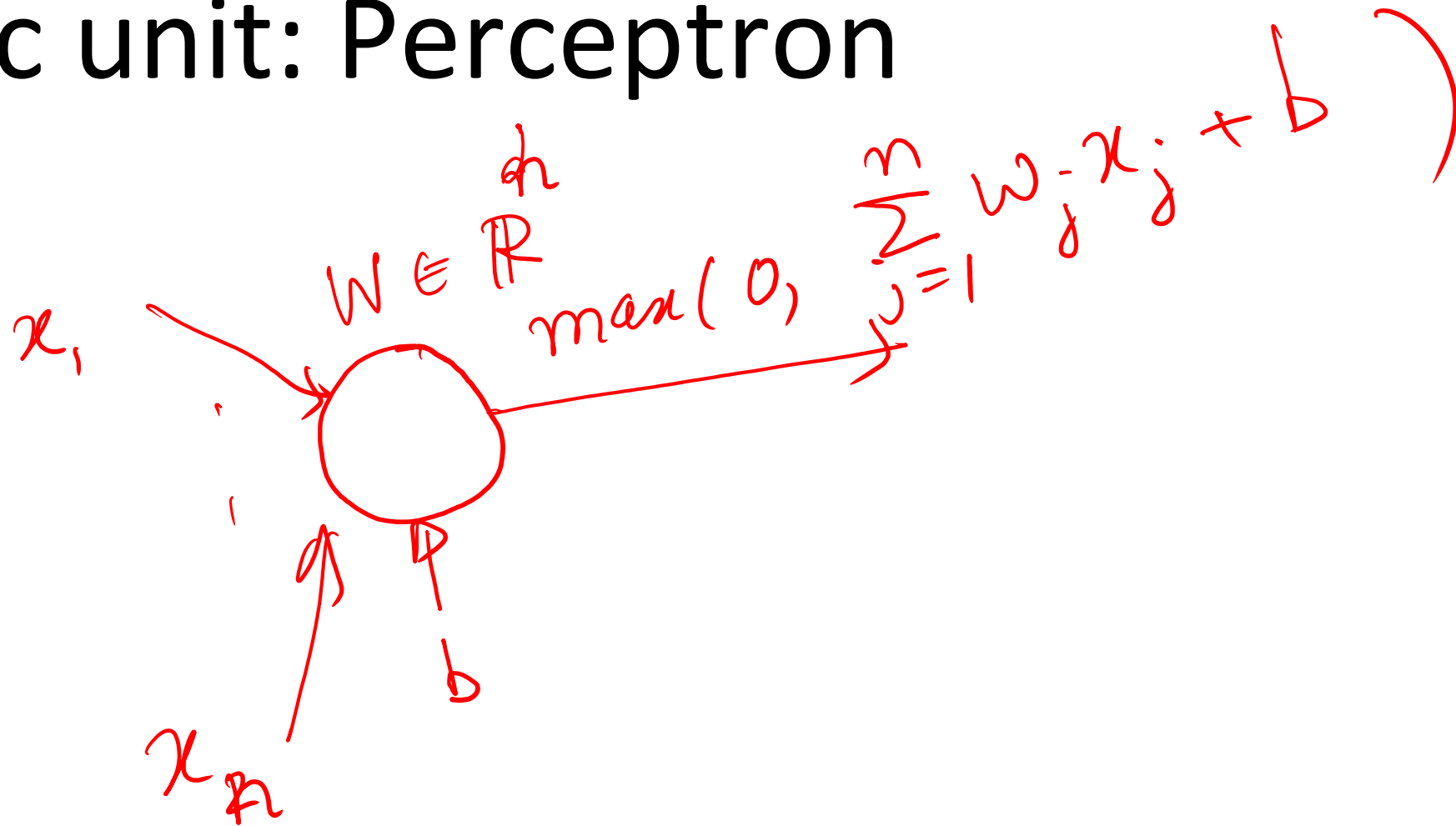


- ★ Universal function approximator
- ★ High-capacity

Feed-forward Neural Network $n = 2$

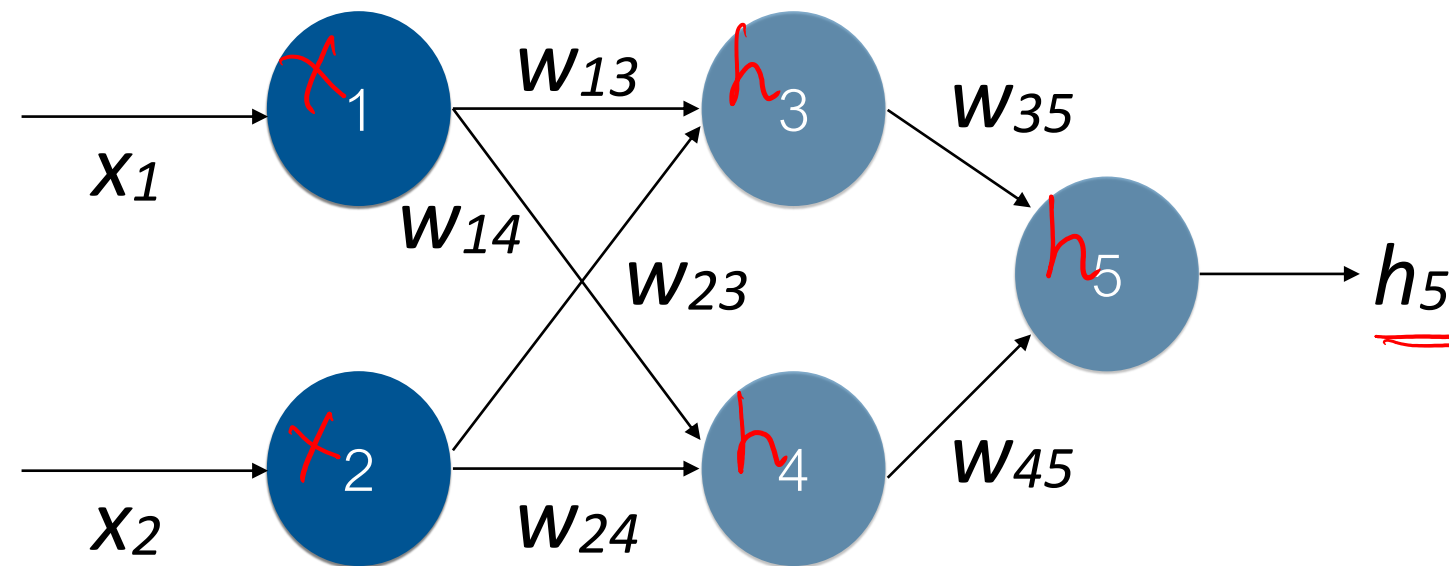


Basic unit: Perceptron



Feed-forward Neural Network

Parameterized Model



$$\begin{aligned} \underline{h_5} &= \underline{g(w_{35} \cdot h_3 + w_{45} \cdot h_4)} \\ &= g(w_{35} \cdot (g(w_{13} \cdot x_1 + w_{23} \cdot x_2)) + \\ &\quad w_{45} \cdot (g(w_{14} \cdot x_1 + w_{24} \cdot x_2))) \end{aligned}$$

Parameters of
the network: all w_{ij}
(and biases not
shown here)

If $\mathbf{x}$ is a 2-dimensional vector and the layer above it is a 2-dimensional vector $\mathbf{h}$, a fully-connected layer is associated with:

$$\mathbf{h} = \mathbf{g}(\mathbf{W}\mathbf{x} + \mathbf{b})$$

where w_{ij} in $\mathbf{W}$ is the weight of the connection between i^{th} neuron in the input row and j^{th} neuron in the first hidden layer and $\mathbf{b}$ is the bias vector

Activation Functions (g)

- Want a function that is efficient to compute, easy to optimize (informative gradient), almost linear
- Cannot be linear: Will get back a linear classifier otherwise

$$h_5 = g(w_{35} \cdot h_3 + w_{45} \cdot h_4)$$

$$= g(w_{35} \cdot (g(w_{13} \cdot x_1 + w_{23} \cdot x_2)) +$$

$$w_{45} \cdot (g(w_{14} \cdot x_1 + w_{24} \cdot x_2)))$$

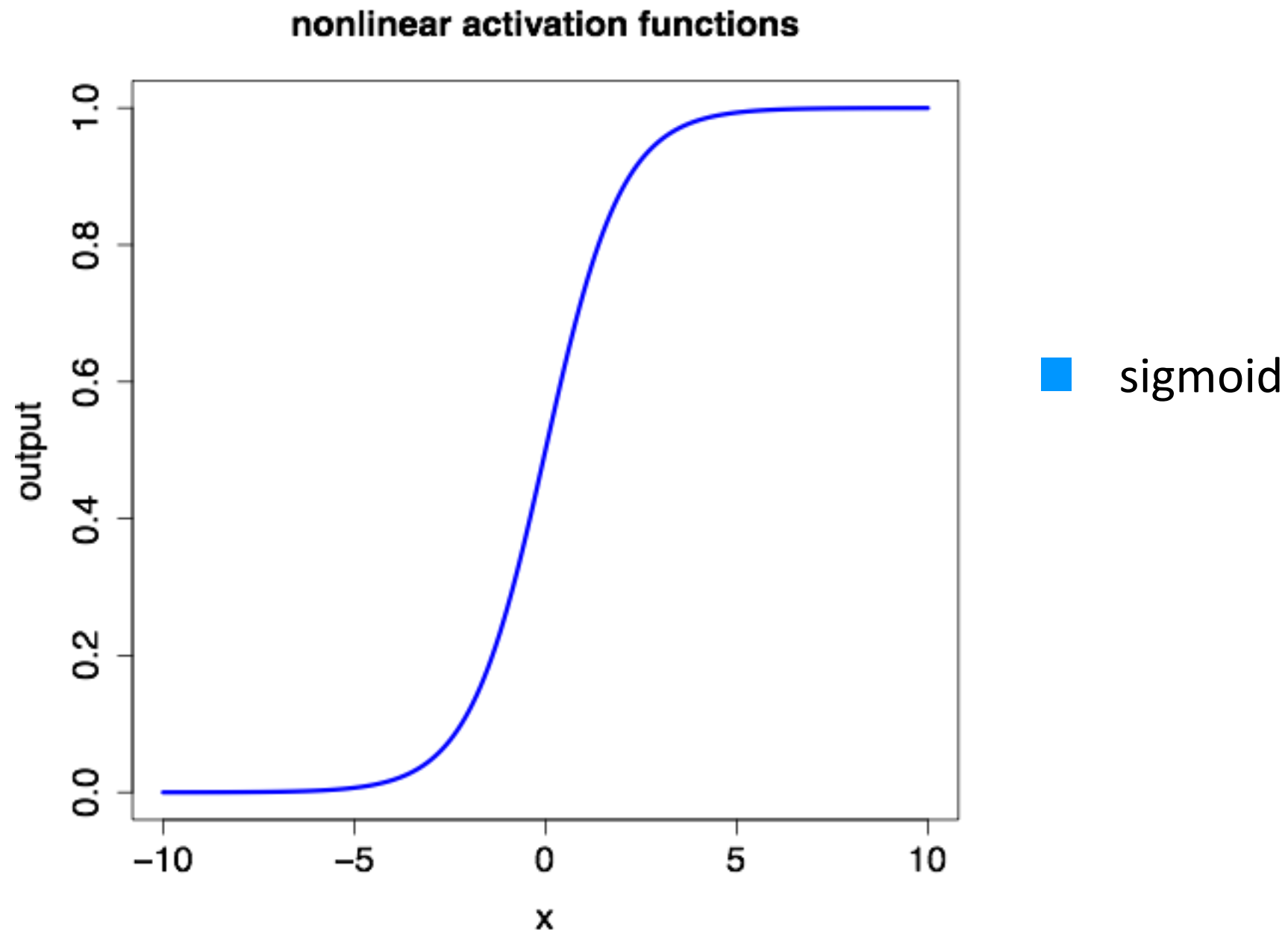
If $g(z)=z$ is a linear function we get:

$$h_5 = (w_{35}w_{13} + w_{45}w_{14})x_1 + (w_{35}w_{23} + w_{45}w_{24})x_2$$

$\underbrace{\qquad\qquad\qquad}_{\tilde{w}_1}x_1 + \underbrace{\qquad\qquad\qquad}_{\tilde{w}_2}x_2$

Common Activation Functions (g)

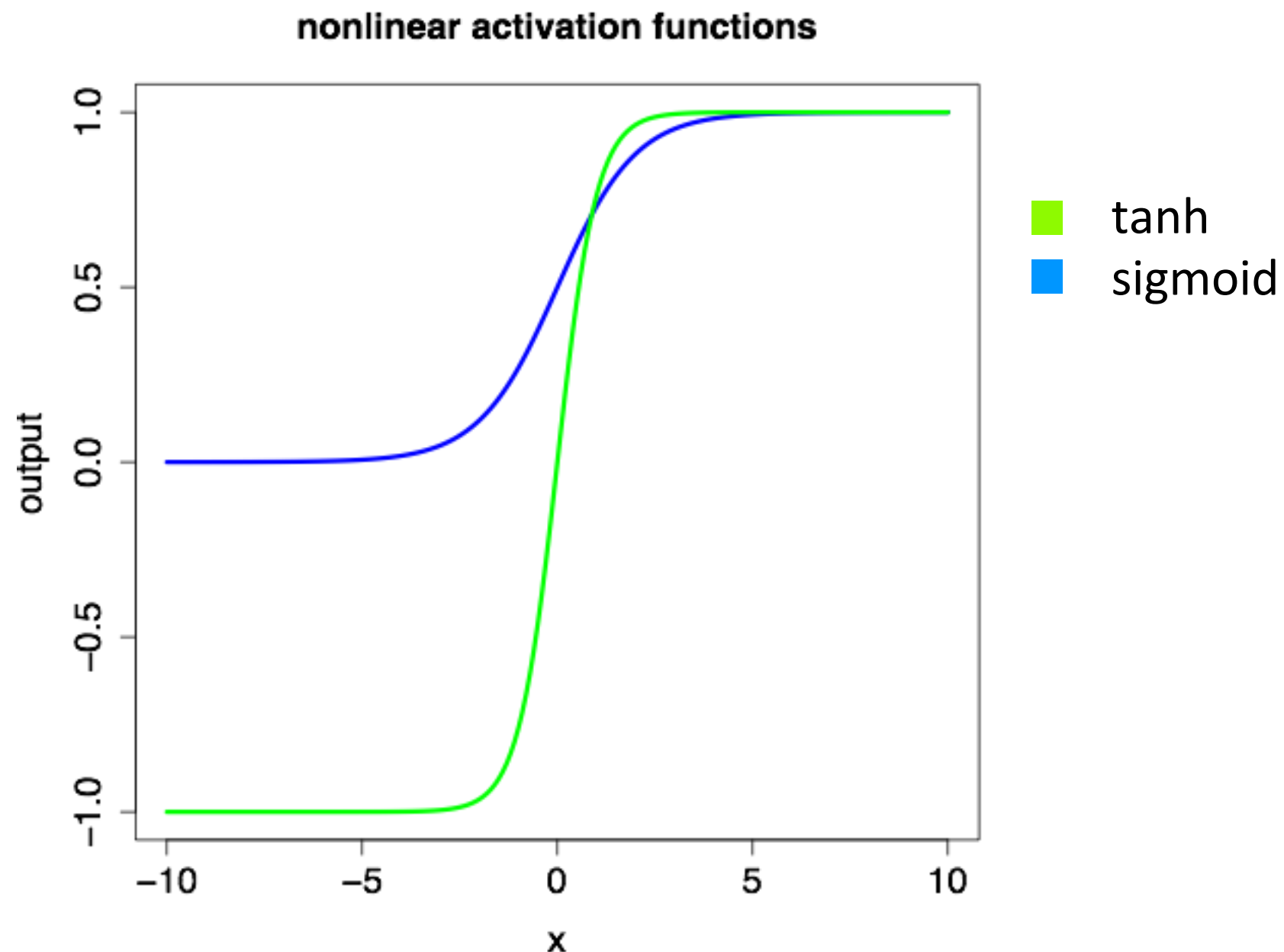
Sigmoid: $\sigma(x) = 1/(1 + e^{-x})$ ← Not convex



Common Activation Functions (g)

Sigmoid: $\sigma(x) = 1/(1 + e^{-x})$ ← **Not convex**

Hyperbolic tangent (tanh): $\tanh(x) = (e^{2x} - 1)/(e^{2x} + 1)$

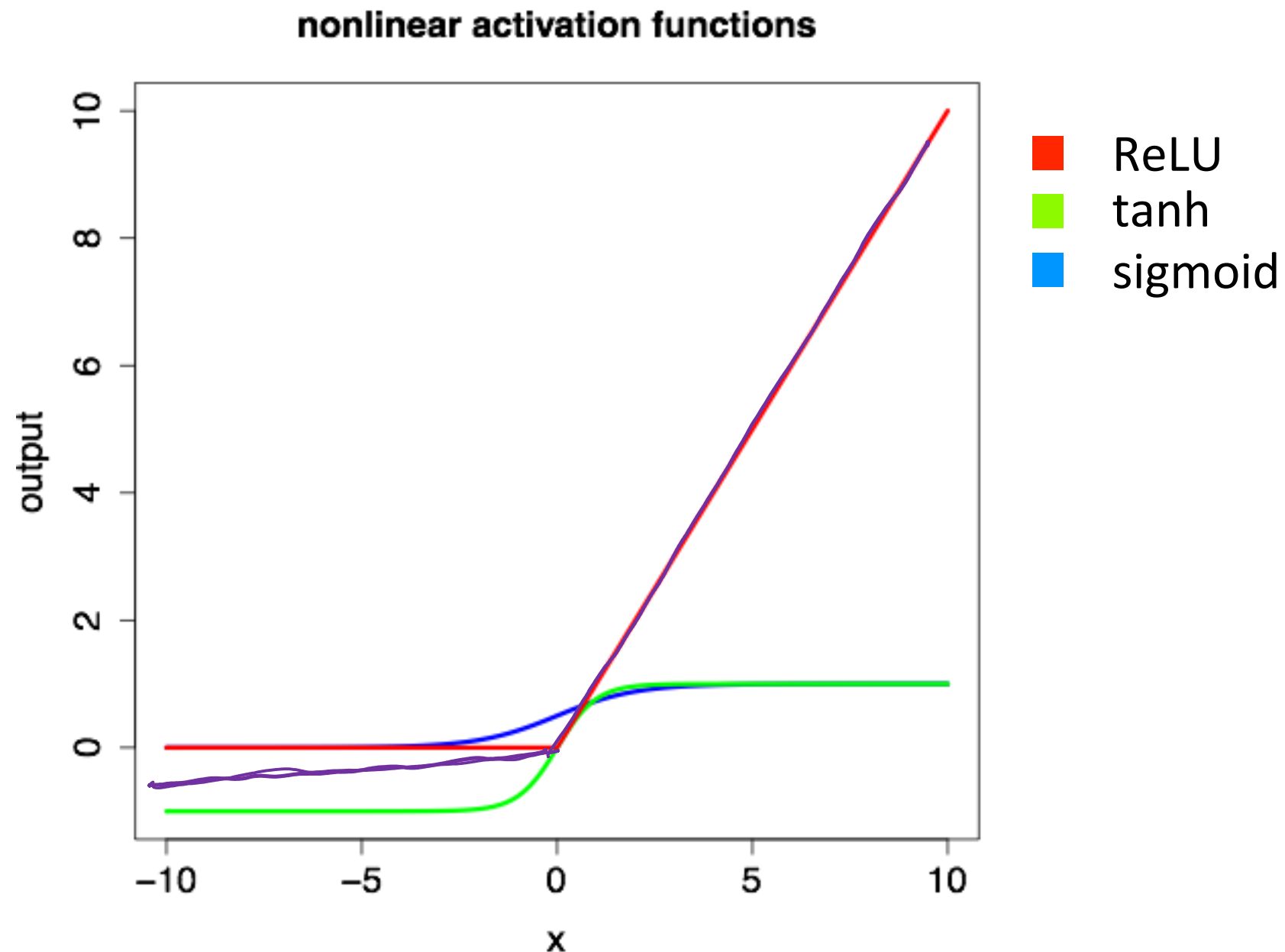


Common Activation Functions (g)

Sigmoid: $\sigma(x) = 1/(1 + e^{-x})$

Hyperbolic tangent (tanh): $\tanh(x) = (e^{2x} - 1)/(e^{2x} + 1)$

Rectified Linear Unit (ReLU): $\text{ReLU}(x) = \max(0, x)$ $\leftarrow$ Convex



Choosing $g()$

Considerations: want some non-linearity, informative gradient (e.g. when convex), fast computation, close to linear

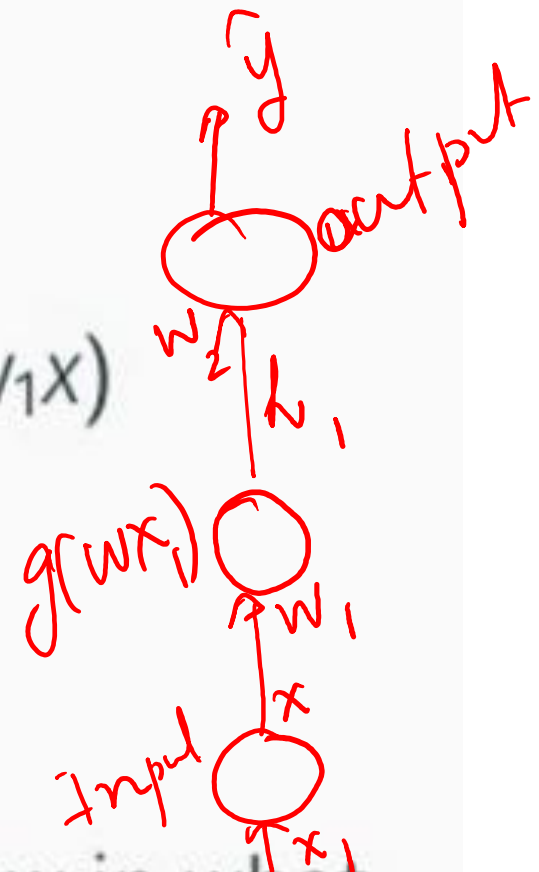
Role of the gradient of g during training

Training objective of DNN with one hidden unit $h = g(w_1x)$

$$J(w_1, w_2, x, y) = L(hw_2y) = L(g(w_1x)w_2y)$$

Gradient of above w.r.t w_1 is $L'w_2yg'x$

If $g' = 0$, the gradient becomes zero and we do not know in what direction to move w_1 .



Choosing $g()$

- RELU: not differential but okay since gradient is informative. second-derivative zero in most places (useful for optimization)
 - Caution: watch out for inactive RelU: initialize affine input bias parameter to small positives. Gradient zero $\Rightarrow$ information flow to lower layers is blocked.
- Sigmoid/Tanh: $\tanh(z) = 2 \text{ sigmoid}(2z)$. Non-convex. Well-behaved (linear) only for small values of z , gradients very small for small or large z , problem for multi-layer network.

Output layers

- Depends on the output type
 - Real: output is mean of Gaussian distribution.
 - Binary class labels: sigmoid function transforms arbitrary reals to probability of Bernoulli
 - Multi-class class labels: softmax provides multinomial probabilities
- Advantage of all of above: Probability distribution over output. Maximum likelihood training loss is convex in parameters of outer-most layer.
- Similar to conventional training.

Neural Network demo

[Playground.tensorflow.org](https://playground.tensorflow.org)

Example XOR

Neural networks can model decisions that conventional linear classifiers cannot.

$$y = f^*(x) = x_1 \oplus x_2$$

Training data = all four combinations.

Linear classifier $\hat{y} = w_1x_1 + w_2x_2 + b$ trained with least square loss yields $w_1 = w_2 = 0, b = 1/2$

Cannot discriminate

Non-linear classifier such as one with x_1x_2 as feature

($\hat{y} = w_1x_1 + w_2x_2 + w_3x_1x_2 + b$) can discriminate but the burden is on us to create the useful non-linear features.

$$w_1 = 1, \quad w_2 = 1, \quad w_3 = -2, \quad b = 0$$
$$\hat{y} = x_1 + x_2 - 2x_1x_2$$

Example: XOR

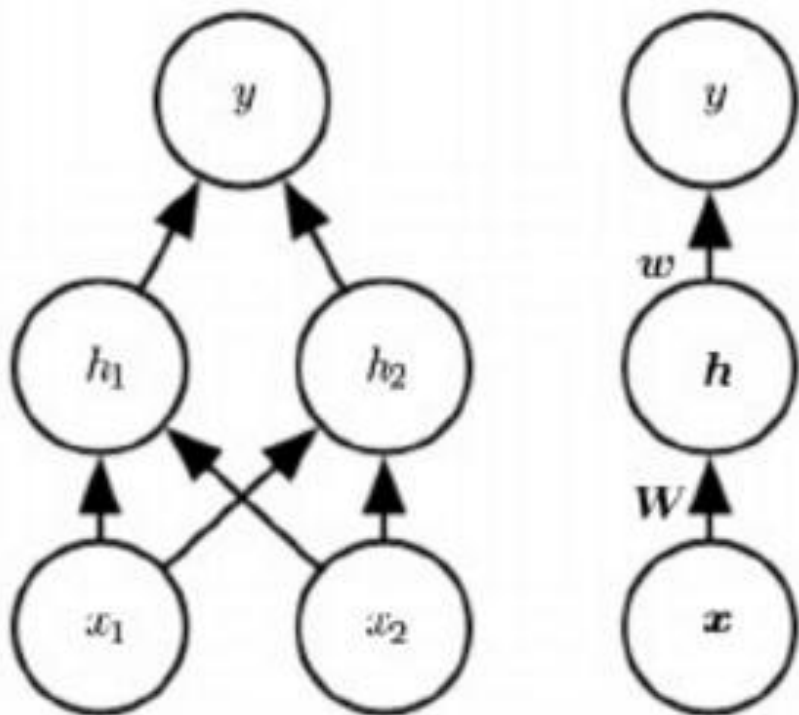
A generic two layer neural network with ReLU:

$$y = f(x) = W^2 \max(0, W^1 x + b^1) + b^2$$

Role of non-linear transform.

$$W^1 = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

$$b^1 = \begin{bmatrix} 0 \\ -1 \end{bmatrix}, W^2 = \begin{bmatrix} 1 \\ -2 \end{bmatrix}, b^2 = 0$$



But what is a neural network? | Chapter 1, Deep
learning - YouTube